database or add a dependency on an existing one. After all, a shared token database is a single point of failure. But the Achilles' heel of stateless tokens is how to handle token revocation. If all the state is on the client, it becomes much harder to invalidate that state to revoke a token. There is no database to delete the token from.

There are a few ways to handle this. Firstly, you could just ignore the problem and not allow tokens to be revoked. If your tokens are short-lived and your API does not handle sensitive data or perform privileged operations, then you might be comfortable with the risk of not letting users explicitly log out. But very few APIs fit this description; almost all data is sensitive to somebody. This leaves several options, almost all of which involve storing some state on the server after all:

- You can add some minimal state to the database that lists a unique ID associated with the token. To revoke a JWT you delete the corresponding record from the database. To validate the JWT you must now also perform a database lookup to check if the unique ID is still in the database. If it is not, then the token has been revoked.
- A twist on the above scheme is to only store the unique ID in the database when the token is revoked, creating a blocklist of revoked tokens. To validate you check to make sure that there isn't a matching record in the database. The unique ID only needs to be blocked until the token expires, at which point it will be invalid anyway. Using short expiry times helps keep the blocklist small.